Emending Nonlinear Recurrence

Erik Garrison
erik.garrison@gmail.com
Poietic PBC / UTHSC, Memphis, TN, USA

Abstract

Fast interaction and feedback require models to update what they know as new inputs arrive. Emender performs this computation in bounded recurrent memory: each input corrects the memory's prediction and applies a nonlinear transformation to the state carried forward. Many small memory heads execute in parallel, making the sequential update practical to train. At 1.3 billion parameters, E97 trains through 150.8 billion Pile tokens on one eight-GPU workstation, reaching training loss comparable to a closely parameter-matched Gated DeltaNet-2 control. An extended Frontier run reaches 513.0 billion cumulative tokens, with its final segment trained on CommaPile. Fine-tuning the workstation pair on an iterated nonlinear map gives E97 99.96% accuracy versus GDN2′s 76.09% at the trained length, and 76.07% versus 53.36% at eight times that length. GDN2 leads on long-horizon S_5 permutation composition, a task that admits an exact linear-state representation. Its strong learning on this task demonstrates the utility of linear-state recurrence, alongside E97′s advantage on repeated nonlinear transformation. Formal results characterize E97′s repeated nonlinear state transformation and GDN's affine state composition under a fixed, state-independent parameter schedule. Together, these findings establish a practical route to training nonlinear recurrence and support further study of computation through memory in feedback, optimization, and driven dynamics.

Keywords recurrent neural networks · language modeling · nonlinear recurrence · state-space models · expressivity · Lean 4

1 Introduction

Keeping balance, following a moving object, and adjusting a movement require continual revision of an internal estimate as observations arrive. Conversation and perception likewise require a model to update what it understands while the interaction unfolds. Fast feedback puts this computation under a per-input time budget. Textual scratchpads and chains of thought let language models perform visible intermediate computations through generated tokens [1], [2]. Each additional token adds a decoding step and extends the working sequence. A complementary question is how much of this ongoing computation can happen directly in carried memory as inputs arrive. Efficient nonlinear recurrence offers such a path: at inference, a fixed-width, fixed-depth model transforms fixed-size state with bounded work per update.

A sequence can require more than a record of what arrived. Its elements may be instructions that change a running calculation, measurements that alter a feedback response, or observations

that revise an earlier association. In each case the current state is part of the next computation. A recurrent model reads an input, updates its internal state, and carries that state into the following step. The question is what computation the update performs, and how to train it efficiently at language-model scale.

Emender uses a nonlinear matrix memory. At each step, a head reads its current prediction at an input-supplied key, writes a correction toward the supplied value, and passes the updated state through tanh. The write uses what the memory already contains; the nonlinearity transforms what the next step will receive. Processing a long sequence therefore executes a long composition of nonlinear state updates.

An *emender* is one recurrent layer, and *Emender* is the family of networks formed by stacking these layers. The current instance, *E97*, uses separate key-axis erase/read and value-axis write gates. Each head has a small matrix state, while projections, gates, and a SwiGLU MLP connect the memories to the rest of the network (Section 3). The memory is revisable: later inputs can correct earlier associations.

This choice creates a systems problem. Modern linear-state recurrences use the composability of affine state updates to parallelize training along time [3], [4], [5], [6]. E97 keeps each head's nonlinear time loop serial and obtains parallelism across heads and batch elements. We call this *width-axis multi-programming*: many small memory programs run side by side, with each state tile kept on the accelerator (Section 4). This complements hybrid recurrent-attention stacks and iterative methods for parallelizing nonlinear recurrence [7], [8].

The first empirical question is whether this route is practical. E97 trains through 150.8 billion tokens on The Pile [9] on one eight-GPU workstation. Its GDN2 control is within 0.01% of its parameter count. Both arms include the same SwiGLU MLP design; we use *GDN2* in the prose for the experimental arm recorded as `GDN2-MLP`. The pair uses the same tokenizer, 2,048-token chunks, optimizer, and local DiLoCo recipe; CMA-ES selects each model's geometry and learning rate. Their training losses converge to the same narrow band at matched tokens (Section 5). The actual-geometry systems preflight also measures comparable throughput and lower peak memory use for E97. The same E97-MLP architecture was then continued on Frontier from the 150.79-billion-token checkpoint to 513.0 billion cumulative accepted tokens, and continued training reduced the recorded rank-0 loss from 2.4365 to 2.2798 nats/token, with the final segment on CommaPile mainmix (Section 5).

The second question is what these trained models can learn to compute. An iterated nonlinear map repeatedly transforms a running value, directly testing the computational pattern that motivates E97. After fine-tuning on a modular-quadratic recurrence, E97 reaches 99.96% versus 76.09% accuracy at the trained length and 76.07% versus 53.36% at eight times that length. Its higher accuracy and greater length robustness support nonlinear transformation in the carried-state path as a useful inductive bias. The S_5 group-state task asks the models to compose permutation actions. This computation admits an exact linear-state representation: each token multiplies a carried permutation matrix by its generator matrix. GDN2 learns this task more effectively in the long-horizon comparison, demonstrating a useful learned capability on a computation compatible with linear-state recurrence. Section 6 gives the protocols, learning trajectories, and length-dependent results for both tasks.

The formal analysis describes the state computations behind this comparison. Lean 4 verifies E97′s time-ordered nonlinear composition and the affine GDN state fold under a fixed, state-

independent parameter schedule. It also relates their split-gated cores and characterizes the affine group actions of an exact S_5 tracker under a token-only schedule and a decoder that separates reachable states (Section 7). An exact construction within GDN2′s production gate and decay ranges is the more specific realization question G3. These statements make the mechanism precise and supply concrete targets for further experiments and proofs.

The resulting program joins a practical architecture to a testable computational hypothesis. Feedback systems, iterative optimization, and driven dynamics provide further settings in which a running state must be transformed repeatedly (Section 8). We proceed from the update and its implementation to the matched language-model comparison and computational probes, then develop the formal results and next questions.

2 Background

The central distinction is where nonlinear computation occurs. Mamba [3] and Gated DeltaNet [5] use nonlinear input processing and gates, and their full networks are nonlinear functions of the sequence. Once a recurrent layer's input-derived transition parameters are fixed, its carried-state update is affine in the previous state. E97 instead transforms the carried state nonlinearly at each step. These choices organize computation across time differently and call for different execution strategies.

A standard Transformer [10] processes a sequence through a fixed stack of attention and nonlinear feed-forward layers. Its layer count is independent of input sequence length. Autoregressive generation reuses that stack in successive decoding steps; generated intermediate tokens extend the serial computation. A nonlinear recurrence adds another nonlinear transformation of its carried state at every input step. This motivates asking which computations a model learns to place in the state update, in its layers, or in a generated working sequence.

Notation

The same state-channel symbols are used in the architecture, applications, and formal sections. Head and time subscripts may be omitted for a single step. A prime denotes the returned state. Matrix products and scalar scaling use juxtaposition; $\odot$ denotes componentwise multiplication, and T in a superscript denotes transpose.

Symbol	*Meaning and source-name correspondence*
S, S_h, S_t, S'	Carried state, head state, time-indexed state, and returned state of E97 and GDN. Lean names this matrix `H`.
N, V, d	Key-axis and value-axis dimensions; square formal state dimension $d = N = V$. Lean key dimension is `K`.
k, q, v	Write key, read query, post-activation value injection.
e, w, g	Key-axis erase/read gate (Lean `b`), value-axis write gate, output gate. b_t in an affine fold is an additive offset, distinct from the erase gate.
δ, r	Value-vector correction and retrieved value in the cell.

Symbol	*Meaning and source-name correspondence*
d_h, λ, D	Production per-head decay, abstract scalar decay, and GDN pre-decay matrix. d_h plays λ's role; the formal E97 fold fixes λ, while production decay depends on the input.
x_t, u_t, θ	Token/layer input, generic input injection, and fixed update-law parameters; v is the cell's value injection, with application-specific inputs defined locally.
t, T, h, ℓ, (i, j)	Step, sequence length, head, layer, and matrix-slot indices. The latching result defines a local threshold T.
ω, η, ε, M, μ	Slot write contribution, saturation slack, post-update slack, perturbation bound, and release margin. The latching result uses δ for a scalar perturbation.
s, r_t, w_t, g_t, z_j, h_t, n_t, x_t	Local states: group element, converter residue, filter coefficients, log-gain, optimization iterate, reservoir, population, and plant. These application-specific coordinates instantiate generic state S; x_t also denotes the iterated-map task state in the probe sections.

Linear-state and nonlinear-state recurrence

The recurrent language model lineage runs from Elman networks [11] and the LSTM [12] through the modern linear-recurrent variants: state-space models such as Mamba and Mamba2 [3], [4], gated linear attention [13], and the delta rule [14]. Many modern designs favor time-axis parallelism via linear-state updates, whereas nonlinear-in-time recurrence generally uses a different execution strategy. Gated DeltaNet-2 [6], the split erase/write-gate extension of Gated DeltaNet [5], supplies the linear-state mixer in the current GDN2 control (Section 5).

A single explicit criterion classifies recurrent architectures. A recurrent layer is *linear-state* if its update can be written $S_t = A_t S_{t-1} + b_t$ with A_t and b_t determined by input-derived features, independently of that layer's previous carried state. Conditioning on these parameters makes the update affine in S_{t-1}. Here the state is expressed in vector coordinates; a matrix state may be vectorized for this criterion. If the previous state S_{t-1} appears nonlinearly in its own update equation, the layer is *nonlinear-state*. The distinction is structural and applies across choices of nonlinear activation.

Linear-state designs admit a chunkwise or scan-based parallel implementation along the time axis because composing $S_T = A_T S_{T-1} + b_T = \ldots$ unfolds into a product of inputs only. This is the reason linear-state recurrence dominates the billion-parameter landscape. Mamba, Mamba2 (SSD), RetNet, GLA, DeltaNet, Gated DeltaNet, RWKV-4/5/6/7, HGRN2, mLSTM, MinGRU/MinLSTM [15] and Griffin's RG-LRU [16] are all linear-state by this criterion. At fixed precision and width, both state regimes are finite-state recognizers. Circuit-complexity results study restricted state-space models under specified arithmetic and transition assumptions [17]. For transformers, log-precision and uniform-circuit characterizations are given by [18] and [19]. These results motivate explicit computational tasks alongside language-modeling loss.

Nonlinear-state recurrences include the classical LSTM and GRU; the sLSTM block of xLSTM [20] (memory-mixing through the hidden state inside exponential gates); M²RNN [7] (matrix state inside tanh with raw-write injection); Titans [21] (MLP memory with online gradient updates); and the architecture introduced here. The Siegelmann–Sontag result [22] concerns

nonlinear recurrent nets under its stated precision and resource assumptions. Barrington's theorem identifies the non-solvable S_5 word problem as an NC^1-complete witness.

Matrix state

Matrix state $S \in \mathbb{R}^{N \times V}$ is shared by several recurrent architectures. Linear-state designs (mLSTM, RWKV-5/6, DeltaNet) already use matrix or expanded states; RetNet's accumulation $S_t = \gamma S_{t-1} + k_t v_t^T$ is matrix-valued. The point of matrix state is that an outer-product update $S_t = ... + k_t \delta_t^T$ provides $O(NV)$ scalars of dynamic state at $O(NV)$ computational cost per token, with content-addressable retrieval via $S^T q$. Matrix state is common to the post-Mamba landscape (Mamba2 with `d_state`, GLA, DeltaNet, RWKV-5/6/7, mLSTM, M²RNN). E97 and GDN2 both use this structure.

The S_5 state-tracking probe

The symmetric group S_5 is the group of permutations of five items; it has 120 elements and is the smallest non-solvable symmetric group. Its *word problem* is the following task: the model reads a sequence of permutations — here, adjacent transpositions, the swaps of two neighboring items — one at a time, and after each one must report the running composition of everything read so far, its *prefix product*. By Barrington's theorem [23] this task is complete for the complexity class NC^1 when exact recognition is required over all words. The experiments measure prefix-product accuracy at specified lengths. The solvable-group control S_3 (permutations of three items, 6 elements) is included to factor out the part of difficulty that comes from prefix tracking *per se* rather than from non-solvability.

Permutation composition has a direct linear-state realization. Choose 5×5 permutation matrices P_s with $P_{sr} = P_s P_r$ and carry the running matrix R_t:

$$R_0 = I_5, \quad R_t = R_{t-1} P_{a_t},$$

where a_t is the incoming adjacent transposition. Then $R_t = P_{a_1 \ldots a_t}$, which identifies the full group element. For each token, right multiplication by its fixed generator matrix is linear in the carried matrix, and remains linear after vectorization. Thus S_5 tracking belongs to the general linear-state class despite the group's noncommutativity and the word problem's NC^1 completeness. This representation explains why learning the task is a meaningful success for a linear-state architecture. Its realization within a particular gated transition family is a separate, more specific question (Section 7, G3 in Section 10).

A dynamics taxonomy of head-types: eigenvalue placement × saturation

The linear/nonlinear hinge above, and the choice of where a head's state may decay, reflect, or oscillate, are two independent axes of a single design grid. The *dynamics* of a head's per-token state map locate it in a two-axis grid:

1. *Eigenvalue placement in the unit disk.* Where the along-key eigenvalue of the per-token state transition sits in the complex unit disk. A real-positive eigenvalue is a *decay* — the stored value fades toward zero along the key. A real-negative eigenvalue is a *reflection* that flips sign each step (`reflect`); reaching it is the lever that lets a recurrence track non-solvable group structure such as S_5 in the restricted transition analyses motivating this grid. A complex-conjugate pair is a *rotation* (`rot`) that oscillates the stored value. Real-positive

and real-negative are the two ends of the *real diameter*; complex placement walks off that diameter into the rest of the disk.

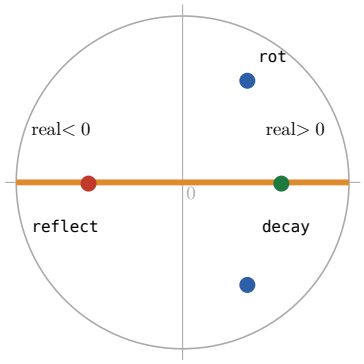
2. *State map: linear or saturated.* Whether the state update is left linear or passed through a bounded saturating nonlinearity (canonically hardtanh; the `-nonlin` axis; the Emender uses the smooth variant tanh). Saturation latches a driven slot and makes the head bounded; finite precision then gives a finite state space.

The grid these two axes span names every Emender head-type (Figure 1). Real × linear at the positive end is `decay`, which is vanilla Gated DeltaNet; at the negative end it is `reflect`, Gated DeltaNet permitted a negative eigenvalue. Real × saturated is `nonlin` — the saturated delta-correcting head, historically tagged E97, whose split-gated instance is the 1.3 B model of this paper with split erase/read and write gates. Complex × linear is `rot`, the complex-eigenvalue oscillator head. Complex × saturated is `rot-nonlin`, a nonlinear oscillator left to future work, included as the combination of the two axes. E97 identifies the trained instance; the architecture vocabulary is this grid.

The adjacent split-gated cores locate the linear-recurrent baseline in this grid.

Shared split-gated cores, with different decay placement. Gated DeltaNet-2 and E97 have adjacent split erase/write cores. E97 uses $\lambda S - k(e \odot k)^T S + k(w \odot v)^T$, whereas GDN-2 uses $(I - k(e \odot k)^T)DS + k(w \odot v)^T$. The exact Lean identity applies E97's unit-decay linear core to the *pre-decayed* state DS; with identity decay the cores coincide (Section 7). The grid organizes these heads by their state dynamics.

A. Eigenvalue placement in the unit disk



Real diameter (orange) = GDN-2; off-axis = rotation.

B. Head-type grid: eigenvalue placement × saturation

	decay (real > 0)	*reflect* (real < 0)	*rot* (complex)
linear	`decay`	`reflect`	`rot`
+ hardtanh (`-nonlin`)	`nonlin`	`nonlin`	`rot-nonlin` (future)

Shading: GDN-2-adjacent linear cores; decay placement differs.

Figure 1: *The Emender head-type taxonomy: eigenvalue placement × saturation. (A)* Each head is fixed by where its along-key eigenvalue sits in the complex unit disk — real-positive `decay`, real-negative `reflect`, or complex `rot` — and by whether the state map is linear or saturated. The orange real diameter is the locus GDN-2 occupies. *(B)* The two axes name five cells. Shading marks GDN-2-adjacent linear cores; the Emender also explores off-axis rotation (`rot`) and saturated state maps (`nonlin`) as a within-layer pool. This taxonomy is descriptive: different decay placement prevents an exact equal-parameter restriction of the production cores. `rot-nonlin` is reserved for future work. The GDN-2 corner of this grid is the mixer of the 1.3 B linear baseline of Section 5 (GDN2), selected by the same search protocol as E97.

3 Computing with a nonlinear memory

Per-head update

Section 1 named the emender layer by what it does — read what the memory predicts at the addressed slot, compute the prediction error, and write the correction — and named E97 as the 1.3 B instance carried through the paper. This section makes the layer concrete: the per-step computation, the choices that fix it, and the shape numbers that make E97 one specific member of the family.

An emender layer runs H independent heads side by side. Each head h owns its own matrix-state memory $S_h \in \mathbb{R}^{N\times V}$ — the matrix state of Section 2, here addressed by an N-dimensional key and holding a V-dimensional value at each addressed slot. (Section 7′s resource theorems use a single symbol d for the common matrix dimension; in E97, $d = N = V = 32$, so each head's memory is a 32×32 tile.) At each token, the layer's input projections supply, per head, a key k_h and query q_h in $\mathbb{R}^N$ that name the slots to write and read, a value $v_h \in \mathbb{R}^V$ to store, a scalar decay $d_h \in (0,1)$ that fades the old state, and an output gate $g_h \in \mathbb{R}^V$. E97 controls its write with two input-dependent gates, split along the two axes of the matrix state: a key-axis *erase/read* gate $e_h \in (0,1)^N$ that scales the address under which the old content is retrieved, and a value-axis *write* gate $w_h \in (0,1)^V$ that scales what is written (the GDN-2-inspired half of the design; Section 7′s split-gated theorem set pins the relationship). The display below is one head's full step in execution order: normalize the two addresses, gate the read address and the write value, read the memory, form the retrieval error, update the bounded state, and gate the read output. The formal core identities in Section 7 use abstract real-valued key/gate/state domains. The production ranges are specified above. This per-head step is the core, bound to the full stack by limitation G2 (Section 10).

$$
\begin{aligned}
k_h &\leftarrow \frac{\mathrm{silu}(k_h)}{\|\mathrm{silu}(k_h)\|_2} \quad (\mathrm{L}^2\text{ -normalized key})\\
q_h &\leftarrow \frac{\mathrm{silu}(q_h)}{\|\mathrm{silu}(q_h)\|_2} \quad (\mathrm{L}^2\text{ -normalized query})\\
e_h &= \sigma(E_h x_t) \quad \text{(key-axis erase/read gate)}\\
w_h &= \sigma(W_h x_t) \quad \text{(value-axis write gate)}\\
r_h &= S_h^T(e_h \odot k_h) \quad \text{(gated read at the key)}\\
\delta_h &= w_h \odot v_h - r_h\\
S_h' &= \tanh\big(d_h S_h + k_h \delta_h^T\big)\\
y_h &= \mathrm{silu}(g_h) \odot (S_h')^T q_h
\end{aligned}
$$

The displayed normalization is idealized for nonzero addresses; production regularizes with `norm+1e-6` (shared fused normalization: `elman/models/e88_fused.py:239-242`).

Here silu is the smooth pointwise activation $x \odot \sigma(x)$, with σ the logistic sigmoid, v_h is the post-activation value input (the activated value supplied to the cell, so the write gate scales an already-activated write), and $\|\cdot\|_2$ is the Euclidean norm. The first two lines put the key and query on the unit sphere so the addressing is bounded; the middle lines make the write a correction against the content already read — under a gated address and at a gated magnitude; and the final line gates only the read output. The state stays inside tanh, so the update is

nonlinear-state for every gate setting. The formal specialization and adjacent-core identities are given in Section 7. This is the per-head primitive. Algorithm 1 spells out the same read-before-write order; its inputs are the projected quantities described above, and its output uses the returned state.

Algorithm 1 — E97 per-head step.

```
Input: S, projected k, q, activated v, d, e, w, g
1. k <- silu(k) / norm2(silu(k))
2. q <- silu(q) / norm2(silu(q))
3. r <- transpose(S) (e hadamard k)
4. delta <- (w hadamard v) - r
5. S_next <- tanh(d S + outer(k, delta))
6. y <- silu(g) hadamard (transpose(S_next) q)
Return: S_next, y
```

E97 repeats the same per-head equation across a residual stack.

E97 fixes these family choices to one billion-parameter instance: model width 1792, depth 11, $H = 216$ heads per layer, $N = V = 32$, expansion 1.0, a bias-free SwiGLU MLP of hidden width 4,032 after each mixer (mlp ratio 2.2623), per-rank batch size 4, a recurrence chunk length of 2048 tokens, and 1,286,589,072 parameters in total (1{,}048{,}152{,}720 non-MLP plus 238{,}436{,}352 in the eleven MLPs). For E97, at depth 11, $H = 216$, and batch size 4 the stack runs $11 \times 216 \times 4 = 9{,}504$ independent recurrent programs per token, each carrying a 32×32 state tile ($216 \times (32 \times 32)$ recurrent-state scalars per layer). Around the recurrent core the language-model wrapper is conventional — a pre-norm residual stack with a final normalization and tied input/output embeddings. Heads never share recurrent state; each layer learns its own key, query, value, decay, split-gate, and output-gate projections; and the recurrent state starts at zero when no prior state is carried in.

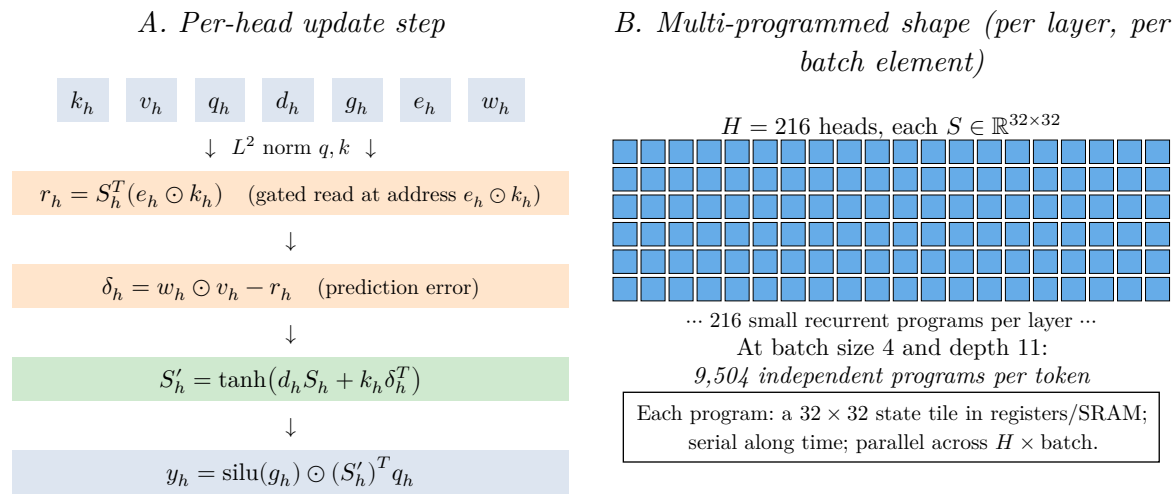


Figure 2: *Emender architecture and multi-programmed shape. (A)* Per-head recurrent dataflow (E97). Each head owns a small bounded associative matrix memory S_h. The model reads under the gated address $e_h \odot k_h$, computes a prediction error δ_h against the gated write value, writes the bounded delta correction into S_h, and gates the read at q_h for output. The tanh on the state, the delta-correcting write, the split erase/write gates, and the L^2-normalization of q, k are visible here; Section 3 enumerates the four load-bearing ingredients of the Emender family. *(B)* The 1.3 B stack exposes 216 small independent heads per layer per batch element. Each head is a 32×32 state tile that fits in registers/SRAM and runs its time loop serially; parallelism is harvested across heads, depth and batch (Section 4), giving $11 \times 216 \times 4 = 9{,}504$ programs per token at batch size 4.

For the Emender family, four ingredients are load-bearing; E97 uses all four in the instance above.

(a) *Bounded matrix state via* tanh *on* S. The state-channel duality (Section 7) distinguishes the executed nonlinear composition from an affine fold. Applying the nonlinearity to the carried state makes an emender layer nonlinear-state.

(b) *Delta-correcting write* $w \odot v - S^T(e \odot k)$. The emender layer reads what memory predicts at the gated address $e \odot k$, computes the prediction error δ against the gated write value $w \odot v$, and writes the correction. Under unit decay and all-one gates, the pre-tanh core with orthonormal keys gives exact overwrite at one slot while preserving orthogonal slots (`linearDeltaWrite_overwrites_one_preserves_others`); full-state tanh, split gates, and arbitrary keys change that guarantee. The current pair shares delta correction; its distinction is the nonlinear carried-state path, together with the stated decay placement.

(c) *Saturation latching of the* tanh *bound.* A slot driven near ± 1 resists perturbations under the saturation theorem's hypotheses; sign-preserving hold and counter-delta release have separate decay and write-budget assumptions. These motivate revisable memory and motivate studying persistence in production E97. Section 7 (saturation insensitivity, sign-preserving hold, counter-delta release) formalizes all three as slot-wise statements on the full emender-layer update.

(d) *Many independent small heads.* E97 at 1.3 B uses $H = 216$ heads of 32×32 each ($11 \times 216 \times 4 = 9\{,\}504$ independent programs per token at batch size 4). Per-head L^2-normalized q, k give many independent addressing programs across the value heads.

GDN2 state channel

Algorithm 2 defines the adjacent GDN-2 linear core used by the formal comparison: decay is applied *before* retrieval and correction, and no tanh is applied to the returned state. Its inputs are an already supplied key, split gates, value, and pre-decay matrix. E97 applies tanh to its updated state. Both trained models also use nonlinear input processing and the SwiGLU MLP described in Section 5. GDN2 retains its own projection processing. Algorithms 1 and 2 present the abstract core; the released baselines add projection, normalization, gate-domain, and residual machinery whose full-stack binding is carried by limitation G2 (Section 10).

Algorithm 2 — GDN-2 split-gated linear core.

```
Input: S, D, k, e, w, v
1. P <- D S
2. r <- transpose(P) (e hadamard k)
3. delta <- (w hadamard v) - r
4. S_next <- P + outer(k, delta)
Return: S_next
```

Parameterization choices

E97 parameterizes the per-head decay d_h in log space, following Mamba2 [4]: each head learns a scalar $A_{\log}$ and a bias δ_{bias} — the code names `A_log` and `dt_bias` — and forms the decay as $d = \exp\big(-\exp\big(A_{\log}\big) \cdot \text{softplus}(\alpha(x) + \delta_{\text{bias}})\big)$, where $\alpha(x)$ is the input-dependent term and σ is again the logistic sigmoid. Both decay parameters are exempt from weight decay, as is standard for such timescale parameters. The recurrence is computed in float32 and cast back to the storage

precision, and the tanh on the state uses the numerically stable form $\tanh(z) = 2\sigma(2z) - 1$ in the fused kernel to avoid overflow at large pre-activations. Three remaining choices are: there is no output normalization inside the recurrent layer — an ablation removed it for a 0.10-nat gain, and the wrapper's block-level pre-norm plus final norm suffice — no short convolution on the input. E97 uses the split erase/read and write gates above; its output gate $\text{silu}(g)$ still acts on the read alone.

The 1.3 B stack

Ingredient (d) sets the 1.3 B stack's shape — many small heads — and the numbers follow from it: each head's 32×32 tile is small enough to live in registers (Section 4), the head count $H = 216$ is large enough to give that many independent addressing programs per layer, and eleven such layers — each followed by its SwiGLU MLP, the channel-mixer that pairs with the token-mixing recurrence — reach the 1.3 B-class budget the paired comparison fixes. The exact values were selected by the per-architecture search of Section 5. At batch size 4, depth 11 and $H = 216$ multiply to $11 \times 216 \times 4 = 9\{,\}504$ independent recurrent programs per token, each carrying its own 32×32 state tile. The reference implementation and the fused Triton recurrence kernel that steps these tiles are identified by the source register and release manifests.

The matched architectural contrast

E97 and GDN2 both use matrix memory, delta correction, split editing gates, and a SwiGLU channel-mixing MLP. The key distinction for the present hypothesis is the operation performed on carried state: E97 applies tanh after its correction, whereas the GDN2 state update is affine when its input-supplied parameters are fixed. Decay placement also differs, as the core identities above specify. The full networks contain nonlinear projections, gates, and MLPs. The comparison centers on the transformation applied to carried state. The matched-pair probes test what the trained architectures learn under the recorded protocol (Section 6); the formal analysis states the narrower update-map facts (Section 7).

4 Making sequential updates practical

Multi-programming: the throughput-enabling design choice

Width-axis multi-programming supplies throughput while each head's time loop remains serial. A linear recurrence earns its throughput on the *time axis*: because $S_t = A_t S_{t-1} + b_t$ unfolds into a product that depends on the inputs alone, the step-by-step loop can be re-expressed as a prefix scan or a chunkwise matrix multiplication over the whole sequence at once. E97 executes its nonlinear state update in a serial time loop. *Multi-programming* obtains throughput on the other axis by replicating the recurrent computation across many independent heads — each with its own small bounded matrix state — and harvests parallelism across those heads, across state tiles, and across batch elements, while the time loop inside each head still runs one step at a time.

The design choice is a decoupling: it frees throughput from the *shape* of the update. The recurrent body must stay a bounded, register-resident per-step map at a fixed matrix-state signature — that is what keeps each head cheap. This implementation supports a programmable nonlinear update body. The cost is paid in per-head serial time, most exposed at long sequence

lengths where the serial loop is longest; the gain is that a nonlinear recurrence can keep a modern accelerator busy with no time-axis scan at all.

The 1.3 B Emender shape under multi-programming

A modern GPU exposes many independent streaming multiprocessors, each able to run a small program out of its own registers and shared memory, and multi-programming is built to fill them. E97, the 1.3 B Emender instance, runs 216 such programs per layer per batch element — one per head — so at depth 11 and batch size 4 the layer stack issues $11 \times 216 \times 4 = 9\{,\}504$ small independent recurrent programs per token, each carrying its own 32×32 state tile small enough to live in registers. Keeping the accelerator busy therefore uses parallelism across memory programs. The paired throughput and memory measurements below test the practical consequence at the actual E97/GDN2 geometries; direct E97 occupancy profiling remains to be measured.

Measured throughput and memory use

The throughput measurements and throughput-budget smokes use NVIDIA CUDA.

At the actual 1.3 B pair geometries, the matched-batch preflight on seven RTX 6000 Ada GPUs under DDP measures E97-MLP at 3,211 tokens/s per GPU and GDN2 at 3,290, both at batch 4. The ratio is 0.976: comparable throughput, with GDN2 slightly faster. E97 uses 28,942 MB peak memory per GPU against 35,715 MB for GDN2. E97 therefore fits batch 6 under DDP, reaching 4,470 tokens/s per GPU, where the control cannot fit that batch (`experiments/preflight_100b/RESULTS.md`). The lower memory use enables the larger-batch throughput gain.

The preflight uses DDP at the actual pair geometries. The long-run training below uses eight independent local-DiLoCo islands. Parameter count, measured throughput, and peak memory are the reported resource comparisons; measuring each implementation's actual FLOPs is a further systems experiment.

Fused Triton recurrence kernel

The forward kernel uses an internal `[T, B, H, *]` layout (time, batch, head, ...) and dispatches one program per `(batch, head_block)`. The `BLOCK_H` parameter is autotuned over $\{1, 2, 4, 8, 16\}$ because at $H \geq 256$ per-program-per-head launch overhead dominates. The state tile lives in registers and shared memory for the duration of the T-step loop. The following pieces are fused into the same Triton program: tanh/silu activations on input projections, L^2 normalization of q, k, the recurrent delta write, and the output gate. Each fusion removes one to two `torch.cuda` launches per layer call.

Sparse-checkpoint backward

The forward kernel saves S only every $K = 16$ steps. The backward kernel processes one K-step segment at a time: it forward-replays the K steps to rebuild per-step S_{t-1} from the saved checkpoint, then walks backward to apply the chain rule. The activation memory used by the kernel is $\frac{T}{K} + 1$ checkpoint slots instead of T. At $K = 16$ this is approximately a $16\times$ shrink, at the cost of a single extra forward pass per segment.

Implementation platform

The recurrence is implemented in Triton [24]. The paired production runs, throughput measurements, and expressivity smokes use NVIDIA CUDA. Porting the pair to ROCm would add a direct test of correctness and performance on that backend.

Distributed training

The 1.3 B-class result reported here was produced on a single eight-GPU workstation, with no cluster and no sequence parallelism — both 150 B-token arms of Section 5 ran the same *local DiLoCo* recipe on that one machine: eight independent islands (one per GPU), schedule-free AdamW [25] inside each island, and plain model-weight averaging across islands every $\tau = 250$ local steps (outer learning rate 1.0, momentum 0 — the conservative plain-average form of DiLoCo [26]). Scaling the same recipe past one machine pairs schedule-free AdamW per island with hierarchical local-SGD averaging: each island is one node of 8 GPUs with intra-island DDP, and inter-island synchronization averages model weights on the same $\tau = 250$ cadence. The structural point holds either way: because the Emender distributes work across programs, the 1.3 B implementation uses ordinary head and batch parallelism while preserving each head's time loop.

ParaRNN [8] offers a different systems route, using an iterative solver to parallelize nonlinear recurrence along time. The present implementation instead retains one serial pass per head and obtains its parallelism across heads and batch elements.

5 Language modeling at 1.3 billion parameters

The systems design earns its place if the model can be trained on a substantial language-modeling workload. We compare E97 with a closely parameter-matched GDN2 control under the same long-run recipe. The result is a narrow training-loss band at matched tokens. The following identities, alignment rules, and search records define that comparison.

The matched pair and its training recipe

The 1.3 B comparison is a *pairing*: one nonlinear-state model and one linear-state model, each selected by its own CMA-ES search, trained from scratch with the shared data, tokenizer, optimizer, and local DiLoCo recipe, and read out at matched aggregate tokens. Geometry and learning rate are selected separately for each arm. The two arms are E97 — the Emender split-edit instance of Section 3, paired with a bias-free SwiGLU MLP per layer — and GDN2 — the official-style Gated DeltaNet-2 [6] mixer with the *same* SwiGLU MLP, bound to external source commit `95709fc`. Holding the channel-mixer (MLP) fixed on both arms makes this a comparison of token mixers — the recurrence.

Training identity	*E97-MLP*	*GDN2-MLP*
Parameters	1,286,589,072	1,286,713,448 (+0.0097%)
Mixer	Emender split-edit delta (Section 3)	Gated DeltaNet-2, 4-token conv
Geometry	dim 1792, depth 11, $H = 216$, $N = V = 32$, mlp ratio 2.2623	dim 2176, depth 12, $H = 30$, expansion 1, mlp ratio 3.2587
Base LR (CMA-selected)	1.007×10^{-3}	4.743×10^{-4}
Data	The Pile, 2,048-token chunks, `p50k_base`	identical
Optimizer	schedule-free AdamW, bf16, per-rank batch 4	identical
DiLoCo	8 islands (one per GPU), $K = 250$, plain average, outer lr 1.0, $\beta = 0$	identical
Tokens per optimizer step	$8 \times 4 \times 2048 = 65\{,\}536$	identical
Run	`emender_E97_1.3B_20260709_084606`	`gdn2_gdn2-mlp_1.3B_20260722_083444`
Final step	2,300,930	2,323,616
Final aggregate tokens	150,793,748,480	152,280,498,176
Final last-100 loss (nats/token)	2.4365	2.4034
Source .pt checkpoint SHA-256	`0239706e1f67e4823008a3a2754894b5b94dc1663580d2e40c1c74f7dd6a72b2`	`ffae61cfeeeff820f469f7d66dd31c60e48c455872ff9102e57b15ad75bad59b`

Table 1: *Training identities of the 1.3 B pair.* Both arms ran the same launcher harness on the same eight-GPU workstation: same data file, chunk size, tokenizer, optimizer, DiLoCo policy, token cadence, and checkpoint cadence (saves every 500 steps, on merge boundaries); the arms differ only in mixer, geometry, and the CMA-selected learning rate. Sources: launch manifests `scripts/launch_emender_8gpu_diloco.sh` and `scripts/launch_gdn2_mlp_8gpu_diloco.sh`; per-run `args.json`/release validation receipts (`docs/EMENDER_DILOCO_OPS_HANDOFF_20260722.md`, `docs/validation/e97-huggingface-dense-base-and-cli-agent-release.md`, `docs/validation/gdn2-huggingface-152b-control-release.md`).

Both arms consumed 65,536 tokens per optimizer step (8 ranks × batch 4 × chunk 2,048, validated from the launch manifests and log deltas), so matched optimizer steps *are* matched aggregate tokens. GDN2 was run 1.49 B tokens past E97′s final step so that the comparison could be read near E97′s terminal authority. The final common logged read-out is five optimizer steps earlier than that checkpoint; no paired loss statistic uses GDN2′s extra tokens.

Algorithm 3 separates selection, training, and comparison. It summarizes the recorded pair protocol; the selection account below identifies the retained winner configurations and the missing first-generation search records. The comparison reports several averaging windows.

Algorithm 3 — pair selection, training, and loss alignment.

```
Input: per-family search bounds, candidate budget, Pile recipe
1. For each family, run CMA-ES with its candidate budget.
   Train candidates; score the recorded late-loss window.
   Retain the winner geometry and learning rate.
2. Train each winner on the common chunk/tokenizer recipe.
   Each island takes local schedule-free AdamW steps.
   Every 250 steps, average parameters (outer LR 1, beta 0).
   Record rank-0 losses and merge-boundary checkpoints.
3. Deduplicate each loss log by optimizer step.
4. Intersect logged step IDs through the common cutoff.
5. Compute raw and declared trailing-window loss differences.
Return: pair identities, aligned losses, windowed differences
```

Training loss at matched tokens

The primary read-out is the training-log loss of both arms aligned on exact common logged steps through step 2,300,925 (150,793,420,800 tokens): 92,037 aligned records, zero E97-only and zero GDN2-only records through the cutoff, no interpolation. This is 327,680 tokens (five optimizer steps) before E97′s terminal checkpoint at step 2,300,930. The table uses the last common logged step.

Statistic at the 150.79 B-token cutoff	*E97*	*GDN2-MLP*	*Δ (GDN2 − E97)*
Raw loss at cutoff step	2.4770	2.4324	−0.0446
80-point trailing smoothed	2.4370	2.4267	−0.0103
Mean, last 100 steps	2.4284	2.4423	+0.0139
Mean, last 1,000 steps	2.4358	2.4273	−0.0085
Early comparison, 5.118 B tokens (smoothed)	2.9300	2.8588	−0.0713

Table 2: *Matched-token loss comparison (nats/token, The Pile).* The common logged cutoff is step 2,300,925, five steps before the E97 checkpoint. At the matched cutoff the two arms sit within 0.010 nats/token smoothed ($\approx 0.4\%$ relative), and the *sign of the gap depends on the averaging window*: GDN2 is lower on the full-trajectory and 1,000-step windows, E97 is lower on the most recent 100-step window. Early in training (5.1 B tokens) GDN2 led by 0.071 nats; the gap closed as training progressed. Source: `docs/GDN2_OPS_SNAPSHOT_20260728.md` (final matched cutoff) and `docs/GDN2_E97_MATCHED_LOSS_COMPARISON_20260723.md` (early comparison); both parse rank-0 logs only, deduplicate by optimizer step, and align on exact common steps.

The two current models reach *comparable training loss* after each family's geometry and learning rate have been selected by search. The nonlinear recurrence trains competitively under this substantial shared workload, with a small difference whose sign varies by averaging window. Under the run's recorded byte normalization (`bytes/token` = 3.783 from the frozen `args.json`), E97′s final-window loss of 2.4365 nats/token corresponds to an estimated *0.93 bits per byte* — sub-1-bpb territory for a purely nonlinear recurrence.

Each arm has one training seed. The quantities above come from training logs; held-out tokenizer-invariant bpb and downstream evaluation of the released pair are pending. The carried-in winner configurations and available search records are described below.

Extended training on Frontier

The 1.2866-billion-parameter E97-MLP model was also continued on Frontier from the released 150.794-billion-token workstation checkpoint to a retained checkpoint at 513.014 billion accepted training tokens. The continuation ran 256 nodes and 2,048 accelerator ranks, per-rank batch size 4 and context length 2,048, schedule-free optimization, and hierarchical DiLoCo averaging every 40 local updates. The configuration strip reads: 256 nodes / 2,048 ranks, batch 4 / context 2,048, $K = 40$ DiLoCo, final segment CommaPile mainmix; the paired workstation study trained on The Pile. Its final recorded rank-0 training-loss average is 2.2798 nats/token over the last 100 logging windows. Continued training reduced the recorded loss stage by stage across the retained checkpoints.

Figure 3 shows E97′s recorded training loss across the workstation and Frontier stages, with the retained GDN2 checkpoint as a reference point. The matched-token comparison is in Table 2.

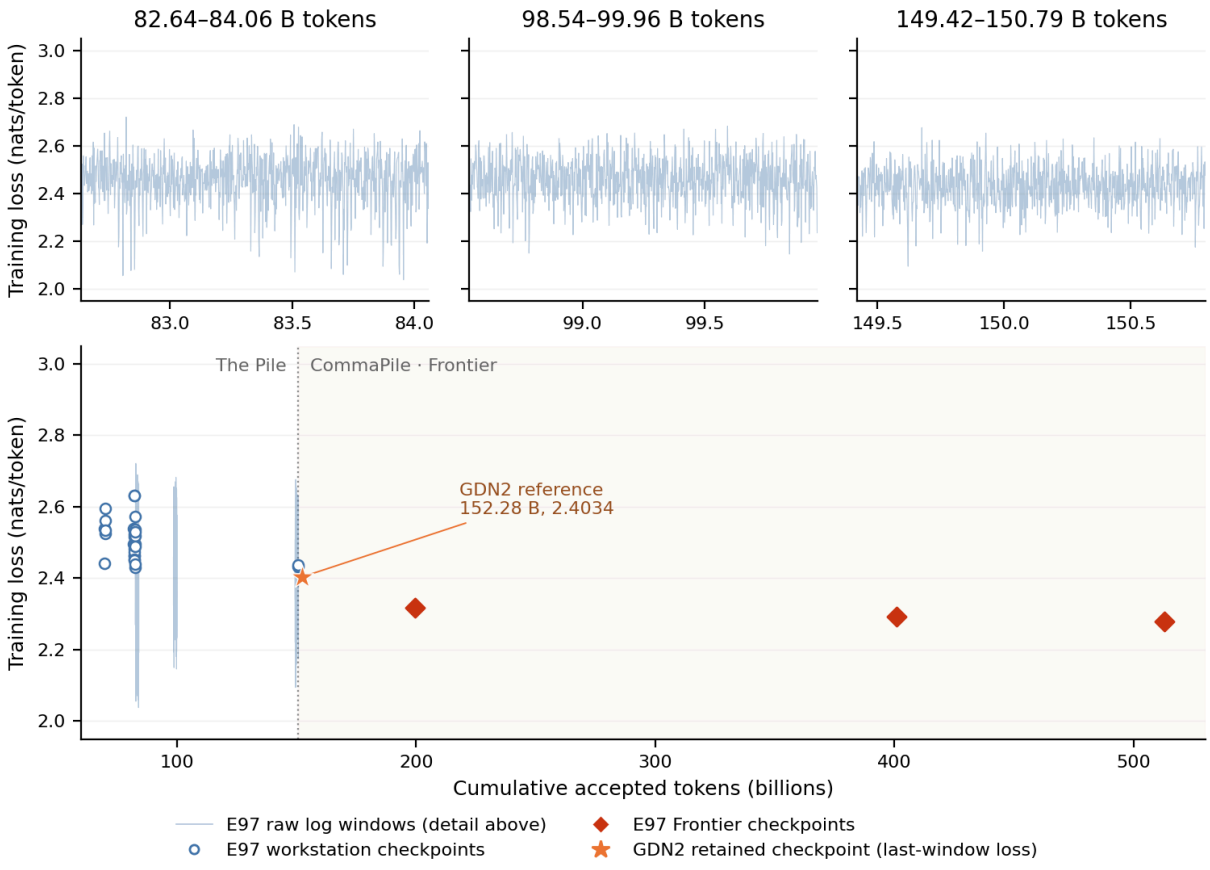


Figure 3: E97 recorded training loss: thin blue traces show three raw log windows (expanded above), circles show workstation checkpoint losses, and diamonds show Frontier checkpoint losses; each trace spans its recorded window and checkpoints appear as separate markers. The orange star is GDN2′s single retained checkpoint at 152.28 B tokens (last-window loss 2.4034), the vertical rule marks the Pile/CommaPile boundary at the 150.794 B-token Frontier handoff, and Table 2 gives the matched-token comparison. Sources and figure script: `paper/figures/plot_recorded_loss.py`.

Stage	*Worker step*	*Accepted tokens*	*Recorded loss*
Released workstation seed	2,300,930	150,793,748,480	2.4365
First 256-node continuation	2,303,840	199,615,447,040	2.3178
Intermediate milestone	2,315,840	400,942,039,040	2.2928
Retained endpoint	2,322,520	513,013,841,920	2.2798

Table 3: *Retained-checkpoint accepted-token lineage of the E97-MLP Frontier continuation (nats/token, rank-0 training log).* The workstation seed contains 150.794 B tokens; a one-time migration receipt binds the first Frontier continuation's legacy checkpoint to 199.615 B tokens, after which the embedded accepted-token counter governs resumption. Each recorded loss is a training-log statistic taken before the schedule-free optimizer's evaluation export. Sources: `docs/validation/e97-final-seed-production-256n-5134243.md`, `docs/validation/e97-total-token-migration-step2303840.json`, `docs/validation/e97-4h-normal-preflight-step2315840.md`, and the corrected 513 B audit.

A verified 256-node production segment initialized all 2,048 ranks from the same released seed and completed 73 hierarchical K40 merges, with median merge duration 6.320 s and median inter-merge interval 72 s. Its last ten logged windows reported a median aggregate throughput of 9.410 million tokens/s. Figure 4 plots those 73 merge durations and the 72 between-merge intervals from the committed verdict data.

Selection of the two models

Both arms were selected by CMA-ES with 15 minutes of Pile training per candidate at 2,048-token context, bf16, fused kernels, and a 1,300 M parameter target with ±3% tolerance. The

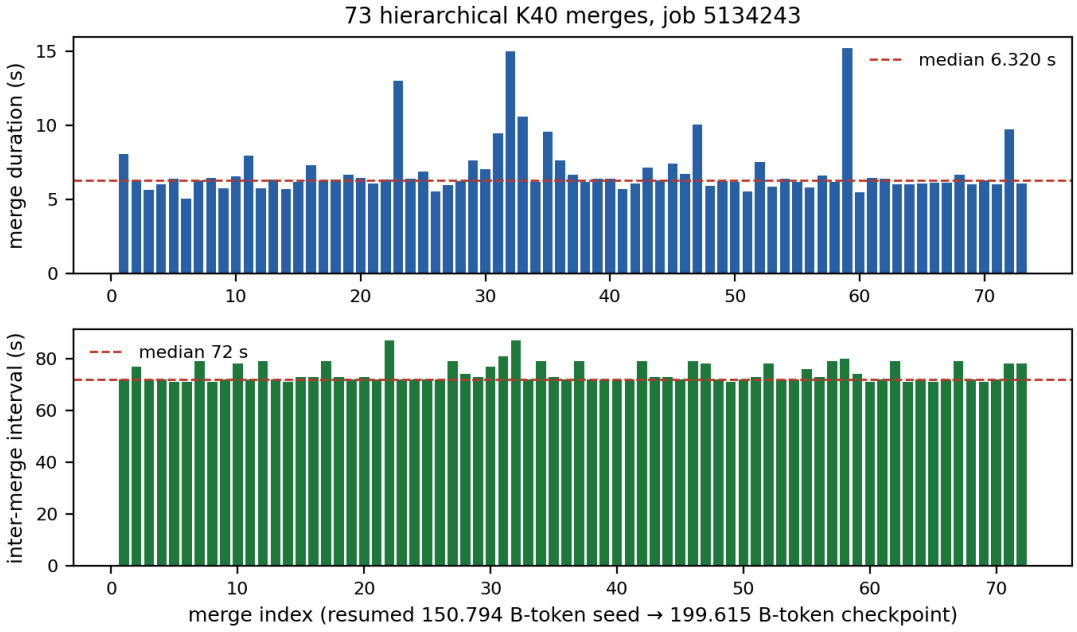


Figure 4: *Hierarchical RCCL communication during the first final-seed 256-node E97 continuation (job 5134243).* Seventy-three $K = 40$ merges had median duration 6.320 s (5.068–15.227 s); the 72 between-merge intervals had median 72 s (71–87 s). The allocation resumed the released 150.794 B-token seed and retained a checkpoint at 199.615 B tokens. These are measurements from one production segment. Source: `reports/frontier/e97-final-seed-production-256n-5134243-verdict.json`; figure script `paper/figures/plot_frontier_segment.py`.

family-specific search selected geometry and learning rate; no subsequent per-arm retuning was applied to either long run.

E97-MLP carries the emender-mlp winner: width 1792, 216 heads, depth 11, state size 32, MLP ratio 2.2623, and learning rate 1.007×10^{-3} (search fitness 5.8606). GDN2 carries the `lb-gdn2-mlp` winner: width 2176, 30 heads, depth 12, expansion 1, MLP ratio 3.2587, and learning rate 4.743×10^{-4} (104 evaluations over 13 generations; best fitness 5.8949 at evaluation 86). The exact configurations and retained search records are in `experiments/lb_compare_20260613/REPRODUCTION.md` and `docs/repro/lb_gdn2_mlp_20260612/best.json`.

First-generation fitness is unavailable for these carried-in winner searches, and E97′s candidate/generation totals are absent from the normalized snapshot. Recovering the original winner-series logs would supply total search effort and within-search improvement for these two configurations (Open item U3.1).

6 What the current models learn to compute

Comparable language-modeling loss leaves open what the carried state can learn to compute. We fine-tune the same 1.3 B E97 and GDN2 checkpoints on two explicit computations. The modular-quadratic task repeatedly applies a nonlinear map to a running value; E97′s advantage supports the mechanism hypothesis that motivates its state update. The S_5 task composes permutations, whose running product admits the linear matrix-state update in Section 2. GDN2′s stronger performance on this task is positive evidence for learning a computation compatible with linear-state recurrence. The two probes therefore test useful learned capabilities with different state-update structure: linear permutation actions and a map nonlinear in its running scalar value. Both studies concern learned performance under stated budgets and protocols.

Iterating a nonlinear map with the trained pair

The decisive experiment uses the E97 and GDN2 checkpoints of Section 5, with Schedule-Free y-mode recovery, on $x_t = (x_{t-1}^2 + c_t) \bmod 256$, 5 input drivers and running supervision. Both arms receive the identical full-fine-tune recipe: bf16 AdamW(betas=0.9,0.95), constant learning rate 0.00005, warmup 300, weight decay 0.01, gradient clip 0.5, batch 32, trained length 128, seed 42. The throughput-derived common cap of 18269 steps was fixed before accuracy was seen (`budget.json`, `1989656c`). It projects 4.35 GPU-hours if both arms reach the cap. The same pre-registered validation-loss plateau rule stopped E97 after 2500 steps and GDN2 after 2200 steps; actual training elapsed 0.649 GPU-hours, excluding loading and final evaluation. The common rule therefore yields different realized training durations.

The protocol below summarizes the retained driver, `scripts/expressivity_1p3b_depthwall.py`, including its zero-based probe indices. Evaluation uses the weights at each arm's plateau stop. The terminal sweep is grad-free and scores every masked position using full-vocabulary argmax; reduced evaluation batches on OOM are recorded with the results.

Algorithm 4 — matched-cap fine-tune and terminal evaluation.

```
Input: bound y-mode weights; common recipe, cap, seed
1. Build fixed validation batches; initialize AdamW.
2. For step j = 0,...,cap-1:
   a. Draw a batch at T=128; run bf16 forward.
```

```
   b. Backpropagate masked cross-entropy over all positions.
      On OOM, split the same batch and accumulate gradients.
   c. Warm up LR for 300 steps, then keep LR constant;
      clip gradient norm to 0.5; take an optimizer step.
   d. Every 100 updates (and at cap), probe validation loss.
      With 4 probes and j >= min(1000,cap), stop if
      loss <= 1e-4 OR both 200/300-step gains are < 1%.
   e. Every 250 updates, record trajectory probes unless stopped.
3. Retain stop weights and actual update count for each arm.
4. For T in {128,256,512,1024,2048,4096,8192}:
   a. Set B=max(1,floor(32768/T)); build 8 seeded batches.
   b. Compute masked accuracy and loss without gradients.
      On OOM, halve B and rebuild with the same length seed.
      If B=1 fails, record an error rather than a score.
Return: training record, stop reason, terminal length sweep
```

Relative gain divides the loss decrease by the earlier loss magnitude (floored at 10^{-12}); both trailing windows must pass. The implemented minimum uses the zero-based index j. Both arms use the stated cap, stopping criterion, and recipe.

Length	*Train-length multiple*	*E97*	*GDN2*
128	1×	99.96%	76.09%
256	2×	99.14%	62.85%
512	4×	90.16%	56.55%
1024	8×	76.07%	53.36%
2048	16×	62.92%	51.91%
4096	32×	55.79%	50.74%
8192	64×	51.79%	50.53%

Table 4: *Modular-quadratic length extrapolation on the released 1.3 B pair.* Full-vocabulary argmax accuracy at the pre-registered plateau stops; trained length 128, seed 42, 262144 scored token positions per length. The common cap is 18269 steps, with unequal realized training: E97 stopped after 2500 steps and GDN2 after 2200. E97 shows graceful length transfer; GDN2 approaches a roughly 50% plateau, above the declared 0.39% random baseline. Source: `paper/results/expressivity_1p3b/iterated-map/`, `ft_{e97,gdn2}_modular_quadratic{,_sweep}.json` (commit `9dc3afd7`); `budget.json` (pre-registration `1989656c`).

At the terminal sweep E97 reaches 99.96% at trained length, retaining 90.16% at four times and 76.07% at eight times that length. GDN2 ends at 76.09% at trained length and 53.36% at eight times. E97 at eight times approximately matches GDN2′s terminal trained-length accuracy, then decays further. GDN2 approaches a roughly 50% plateau at the longest lengths, above the declared 0.39% random baseline. Intermediate GDN2 trajectory probes had higher accuracy than its terminal checkpoint. These single-seed trajectories motivate repeats that compare fixed training budgets and checkpoint-selection rules.

The modular-quadratic finding is consistent with the temporal-composition contrast in Section 7: E97 executes one nonlinear state update per step, while a GDN fold conditioned on an input-only schedule is affine in its initial state. The second probe tests permutation composition, for which a linear transformation of carried matrix state already implements each required update.

S5 group-state tracking on the same pair

The S_5 word problem tests repeated composition of permutation actions. Its exact permutation-matrix representation uses a token-selected linear map at every step (Section 2). This makes S_5 a direct test of learning a computation within the general linear-state class. The modular-quadratic probe instead repeatedly transforms its running scalar through a nonlinear map, testing the inductive bias provided by E97′s nonlinear state path.

Zero-shot evaluation did not separate the two models. Matched fine-tuning favored E97 on the solvable S_3 control. In the longer-horizon S_5 capacity study, GDN2 is ahead of E97 at every reported length, and both curves decline toward the random baseline. GDN2 learns this group-state computation more effectively under the recorded protocol. This is a substantive success of the linear-state model on a task admitting linear state updates. Sources: `paper/results/expressivity_1p3b/SUMMARY.md` (`5e1e60ef`) and `capacity/SUMMARY.md` (`6f916843`).

Length	*E97*	*GDN2*
256	0.397	0.685
512*	0.204	0.363
1024	0.109	0.190
2048	0.061	0.103
4096	0.033	0.054
8192	0.021	0.034

Table 5: *S5 capacity/long-horizon accuracy on the released pair.* Full-vocabulary argmax accuracy, seed 42. GDN2 leads at every reported length on permutation composition, a task admitting linear state updates; both curves decay. *GDN2 completed one optimizer step at length 512 before the curriculum stage proved OOM-infeasible even at sub-batch 1; the length-512 point includes that completed update. Source: `capacity/cap_{e97,gdn2}_s5_sweep.json` and `cap_gdn2_s5_train.json` under `paper/results/expressivity_1p3b/` (E97 `f1ed5615`, GDN2 salvage `6f916843`).

Both capacity curves decline with length; held-out transitions remain weaker than seen ones. GDN2′s attempted length-512 curriculum stage was OOM-infeasible even at sub-batch 1, after one completed optimizer step. The next comparison should use a curriculum feasible for both models and additional seeds.

Parameter-matched probes at 8 M

The two architectures were also compared at 8 M parameters, where a parameter-matched sweep trains each model from scratch under its reasonable-default configuration (dim 384, depth 4, $H = 32$, $N = 32$, schedule-free AdamW, three seeds; GDN uses dim 640 to match parameter count). At S_5 training length ($T = 128$) the Emender reaches 0.7918, GDN 0.3552, and raw-write M²RNN-CMA 0.2157. The Emender starts at 0.79 and declines to 0.42, 0.22, and 0.11 at $T = 256, 512, 1024$. On the solvable S_3 control the Emender is at 1.0000 and GDN at 0.7185, and both raw-write variants stall in the 0.31–0.38 band. Each family runs its default configuration with no probe-specific hyperparameter search, which is the seeds-controlled companion to the current-pair measurements above.

Model	S_3 *T=128*	S_5 *T=128*	S_5 *T=256*	S_5 *T=512*
Emender	*1.0000*	*0.7918*	*0.4158*	*0.2150*
GDN	0.7185	0.3552	0.1843	0.0974
M²RNN-CMA	0.3124	0.2157	0.1120	0.0593
M²RNN-paper	0.3773	0.1698	0.0884	0.0488
random	0.1667	0.0083	0.0083	0.0083

Table 6: *State-tracking accuracy on the permutation-composition probes at parameter-matched 8 M scale.* Mean over three seeds. S_3 is the solvable-group control; S_5 is the non-solvable NC^1 witness. The Emender separates from the linear-recurrent (GDN) and raw-write nonlinear matrix RNN (M²RNN) baselines at training length. Source numbers in `paper/ndmpapernotes.md` lines 153–173; figure script `paper/figures/plot_expressivity_seeds.py`.

Interpreting the current-pair evidence

The iterated-map result is the direct empirical test of the motivating mechanism: E97′s nonlinear state path repeatedly transforms the value that will enter the next update. Its measured advantage is consistent with that hypothesis, while optimization, geometry, and checkpoint selection remain possible contributors. The S_5 result demonstrates that GDN2 learns permutation composition strongly, as measured by its lead at every reported length. The task has an exact linear-state representation, so this result establishes a useful learned capability on a computation of that class. It complements E97′s advantage on iterated nonlinear maps and the pair's comparable language-modeling loss. Together, these measurements make the useful question concrete: which computations benefit from placing the nonlinearity directly in the temporal state path, and under what training and resource conditions?

7 What the update rules establish

The formal results describe the state computations tested above: time-ordered nonlinear composition, the conditioned affine fold, finite-group tracking, adjacent split-gated cores, and conditional latching. Each statement below gives its assumptions and Lean source name.

Temporal composition and routing

The concrete repeated-map family is the schedule of `CompositionalDepthDuality.E97Step` values (k, e, w, v) at fixed λ (Lean field `b` is the paper's erase gate e), acting through `e97Step` on matrix state. `e97Step_eq_tanh_comp` identifies the step as elementwise tanh after the state-affine split-gated core. `e97StateAfter_eq_compose` gives its time-ordered composition, and `e97_compositional_depth_grows_in_time` states that appending a step applies one more such map. Thus the *executed* composition contains one tanh per token.

For a fixed, state-independent schedule of `CompositionalDepthDuality.GdnStepParams`, `gdn_state_channel_depth_bounded_in_time` gives matrices A, C with $S_T = AS_0 + C$ for every initial state (Lean writes H for this matrix). The theorem's state-channel depth is zero under this fixed, state-independent schedule.

The permutation-matrix construction in Section 2 gives an exact S_5 tracker in the general linear-state class. The Lean bridge characterizes realizers in the formal GDN family under explicit assumptions. `S5GDNBridge.gdn_s5_realization_requires_per_token_action` starts from

zero state with a token-only parameter schedule and a decoder that recovers `S5Tracker.run` on every word. It also assumes that equal decoded outputs imply equal reachable states. It then gives a state map F through the group element, with affine word actions implementing right multiplication on the reachable image. `gdn_s5_constant_parameters_cannot_realize` rules out a token-independent schedule. The input token selects the action; the carried state accumulates the running group product through that action. The bridge is a necessary-condition theorem for an assumed exact realizer. Constructing such a realizer within GDN2′s production gate and decay ranges remains G3 in Section 10. The S_5 word problem is NC^1-complete. The complexity separation $\mathrm{TC}^0 \neq \mathrm{NC}^1$ remains open, and no result here depends on it.

The finite-state tracker

At fixed width and precision the Emender is a finite-state machine, and the S_5 word problem it is tested on is a concrete, fully specified finite-state target.

- *Finite-state ceiling at fixed precision.* At fixed width and precision, every online recognizer of this kind has only finitely many distinct states (`S5Witness.fixed_precision_state_space_finite`). This gives a finite online state space.
- *S_5 word problem.* The target is pinned down exactly in Lean. The group S_5 has 120 elements (`S5Witness.s5_state_count`) and is non-solvable (`S5Witness.s5_not_solvable`); the prefix-product tracker that reads a word of transpositions and reports the running product is a 120-state recognizer (`S5Tracker.recognizer_state_count`) whose state composes correctly as tokens are appended (`S5Tracker.run_append`). A bridge lemma confirms the Lean tracker agrees with the Python evaluation harness on every input (`S5Tracker.pythonRun_eq_tracker_tuple`). The four adjacent transpositions generate all of S_5: the tracker's run map is surjective (`S5Inseparability.run_surjective`, via Mathlib's `Equiv.Perm.mclosure_swap_castSucc_succ`).
- *Capacity lower bound (recognizer floor).* Any finite-state recognizer with a state decoder that reproduces the tracker on every input word needs at least 120 states (`S5Inseparability.capacity_lower_bound_120`); equivalently, no recognizer with fewer than 120 states can realize it under any decoder (`S5Inseparability.small_recognizer_cannot_realize_s5`). The bound has a distinguishing-pair form: any recognizer with too few states admits two words it cannot tell apart that the tracker does distinguish (`S5Inseparability.exists_distinguishing_word_pair`). This is the existential form of the S_5-coset inseparability target: it holds for *any* finite recognizer family, with no architectural assumptions.
- *Lookup-table realization.* The tracker is small enough to write down as a table: it uses exactly $120 \times 4 = 480$ state/input keys (`S5NDMRealization.s5_transition_key_count`), and any finite recognizer admits such an exact lookup-table realization (`S5NDMRealization.exactTransitionMemory_run`).

Generator memory and recurrent realization

The exact statement of `NDMRealizesS5.emender_realizes_s5_tracker` supplies $d = 12$, four orthonormal keys and one-hot values, queries, a decoder, and $\lambda = 1$. Reading a *preloaded* table retrieves the current generator; decoding it and *externally multiplying*

a supplied S_5 state s reproduces the transition $s \cdot g$. The linear readout supplies the generator value, and the external multiplication updates the group state. The supporting `OnlineMemory.linearDeltaWrite_overwrites_one_preserves_others` is an exact *pre*-tanh overwrite statement.

A recurrent realization would carry the running group product in its own state. Open item O1 asks for a configuration of the nonlinear update, an initial state and decoder, and an all-words trajectory proof realizing that computation.

Split-gated E97 and GDN-2 cores

The module `SplitGatedDelta` identifies adjacent split-gated linear cores with different decay placement. The identities are over abstract real-valued gate/key/state signatures.

- *Adjacent split-gated cores with different decay placement.* The GDN-2 linear core is the E97 linear core applied to a pre-decayed state (`gdn2LinearCore_eq_e97LinearCore_on_decayed_state`), and under identity decay the two coincide (`gdn2LinearCore_identity_decay_eq_e97LinearCore_one`); the resource signatures then split exactly on the axis the paper tests empirically: E97 has matrix state, full-state temporal nonlinearity, delta writes, and no scan-compatible state path (`e97_split_gated_resource_signature_like_e88`), while the GDN-2-style split erase/write resource has matrix state and delta-style writes but *no* temporal state nonlinearity and a scan-compatible linear state (`gdn2_split_erase_write_is_scan_compatible_linear_state`).

The current pair therefore shares adjacent split-gated delta cores and differs on the nonlinear state path and decay placement, as well as geometry and input processing in the trained stacks. This identifies the formal structural distinction relevant to the iterated-map hypothesis.

Matched-signature delta-versus-raw-write separation

A Lean 4 trusted core machine-verifies that no fixed-weight raw-write matrix RNN with row, column, or cell forget gates matches the Emender's delta-correcting update in one step (`RecurrentResourceFormalism.emender_m2rnn_one_step_resource_separation_embeds`), and that the gap persists for every finite k under composition on a constructed witness alphabet (`emender_m2rnn_k_step_separation`). Exact non-equality holds on that real-valued witness for each finite k. The statement is a representability result inside the formalized fixed-weight raw-write resource class, over abstract real-valued gate/key/state signatures; the probes of Section 6 measure trained-model accuracy.

Conditional latching

Saturation, hold, and release are described by slot-level bounds.

These three results formalize the latching half of the Emender primitive: saturation makes a slot insensitive to further bounded writes, sign is preserved under sub-threshold counter-input, and a sufficient counter-delta releases the latch. All three are slot-wise statements on the full abstract Emender update, where v denotes the bounded post-nonlinearity value input already supplied to the recurrent body, $S' = \tanh\Big(\lambda S + k(v - S^T k)^T\Big)$ at slot (i, j), with $\omega := k_i\Big(v_j - (S^T k)_j\Big)$ the write contribution into that slot. The proofs run through the standard tanh-perturbation

foundation. The module carrying them (`EmenderLatching`, introduced at `696299b4`) is included in the refreshed inventory and the current clean `lake build ElmanProofs.PaperCore`.

- *Saturation insensitivity.* If the slot is saturated in the sense $|S'_{i,j}| > 1 - \varepsilon$, and a perturbation δ to the pre-activation entry has $|\delta| \le M$ with $\varepsilon + M < 1$, then the post-tanh slot moves by at most $2(\varepsilon + M)\,|\delta|$. The bound shrinks linearly in both the saturation slack ε and the perturbation magnitude M; at deep saturation a bounded write barely moves the slot. Lean: `EmenderLatching.emender_saturation_insensitivity`. A two-input corollary `EmenderLatching.emender_saturation_two_inputs_close` lifts the bound to two value vectors sharing one key.
- *Sign-preserving hold.* With $\lambda \ge 1$ and a bounded write $|\omega| \le T < \lambda(1-\eta)$ at slot (i,j), a slot already saturated with $|S_{i,j}| > 1 - \eta$ keeps its sign after one Emender step and stays bounded below in magnitude by $\tanh(\lambda(1-\eta) - T)$. For $\lambda > 1$ and small η, T the argument $\lambda(1-\eta) - T$ exceeds 1, so the lower bound exceeds $\tanh 1 \approx 0.76$ and the latch holds with margin. The hypothesis $T < \lambda(1-\eta)$ is the "no sufficient counter-delta" clause; counter-delta release is the next result. Lean: `EmenderLatching.emender_latch_holds_by_default`.
- *Counter-delta release.* With $\lambda > 0$, $S_{i,j} > 0$, and a write contribution $\omega < -\lambda S_{i,j}$ in the opposite sign, the post-update slot is strictly negative — the latch flips out of its basin in one step. Symmetrically for the negative basin. The release threshold is $T^* := \lambda\,|S_{i,j}|$; any counter-delta exceeding T^* in the opposite sign suffices. Lean: `EmenderLatching.emender_latch_releases_on_counter_delta`. The quantitative companion `EmenderLatching.emender_latch_release_quantitative` shows that if the counter-delta exceeds the threshold by margin $\mu > 0$, the post-update slot lies on the opposite side of the origin with magnitude at least $\tanh \mu$.

Verification of the formal artifact

A trusted Lean 4 [27] core built on Mathlib [28] carries the formal spine: each result is stated as a theorem and compiled together with a proof of it. The import closure of the `ElmanProofs.PaperCore` module — 17 project source files in the recursive local closure — uses none of Lean's escape hatches: no `sorry`, `admit`, added `axiom`, `opaque` declarations, or `native_decide`. The gate is `formal/lean/scripts/check_paper_core.sh`; compiling `ElmanProofs.PaperCore` additionally checks elaboration of the proofs. The refreshed machine-extracted inventory is committed at `paper/data/lean-trust-gate-theorems.json` (857 named declarations, 723 user-written, 17 files; extraction source commit `e58ae937`), including the split-gated, latching, duality, routing, Lipschitz, and E1H-definition modules. Both the gate and extractor traverse the recursive local imports rather than a selected module allowlist. The checked project sources use Lean and Mathlib as their proof foundation.

8 Repeated nonlinear maps in the wild

Balance, motion tracking, and movement correction offer an intuitive starting point: each new observation revises an estimate that guides the next action. The iterated-map study isolates a related computational question: how well can a recurrent model learn to keep transforming an evolving state? A common description is

$$S_{t+1} = F_{\theta(S_t, u_t)},$$

where θ fixes the update law and u_t supplies the input at each step. E97′s all-one split-gate specialization is

$$S' = \tanh\Big(\lambda S + k(v - S^T k)^T\Big)$$

with state S and input (λ, k, v). Its returned state becomes the next step's input. The abstract E97 fold fixes λ and supplies (k, e, w, v) at each step; the production architecture also supplies input-dependent decay.

Algorithm 5 — repeated-map fold.

```
Input: initial state S_0, fixed update law F_theta, input schedule
1. S <- S_0
2. For each input u in time order:
   S <- F_theta(S, u)
Return: S
```

The examples below move from feedback and signal processing to optimization, conversation, and biological dynamics. They suggest further tests of learned recurrent surrogates. For each one, the state equation identifies the computation, and the proposed comparison identifies the regime and error measure that would make a difference useful.

Feedback and movement

A controller must update its estimate and response while a system moves. A discrete-time plant with a smooth actuator limit and state feedback can be written

$$a_t = a_{\max} \tanh\left(\frac{Kx_t + Lr_t}{a_{\max}}\right), \quad x_{t+1} = Ax_t + Ba_t + Ed_t.$$

The state is x_t and the inputs are reference r_t and disturbance d_t. The resulting closed-loop map is $F(x,(r,d)) = Ax + Ba_{\max}\tanh\Big(\frac{Kx+Lr}{a_{\max}}\Big) + Ed$. Actuator saturation is a standard nonlinear-control concern [29]. A learned surrogate comparison could vary how often saturation determines subsequent decisions, measuring trajectory error, latency, and closed-loop stability across operating regimes.

Signal processing

Sigma–delta conversion

A first-order sigma–delta modulator accumulates an analog sample and feeds back a quantized output [30]:

$$a_t = r_t + x_t, \quad q_t = Q(a_t), \quad r_{t+1} = a_t - q_t.$$

The residue r_t is the state, x_t is the sample, and the fixed map is $F(r,x) = r + x - Q(r+x)$. A one-bit quantizer selects between two output levels. Its discontinuous output changes the residue used at the next step. A useful surrogate test would compare exact bitstream accuracy and filtered signal quality near quantizer thresholds over long streams, varying state precision and representation.

Adaptive filtering and gain control

Normalized LMS [31] updates coefficient state w_t using regressor x_t and desired sample d_t:

$$e_t = d_t - w_t^T x_t, \quad w_{t+1} = w_t + \mu x_t \frac{e_t}{\varepsilon + \|x_t\|^2}.$$

For fixed inputs this update is affine in w_t. Adding a componentwise coefficient limit gives $w_{t+1} = \operatorname{clip}_{B\left(w_t+\mu x_t \frac{e_t}{\varepsilon+\|x_t\|^2}\right)}$, which applies a nonlinear state transformation after the correction.

Automatic gain control feeds measured output level back into gain [32]. A bounded log-gain controller can use

$$y_t = \exp(g_t)x_t, \quad g_{t+1} = \operatorname{clip}_{[g_{\min},g_{\max}]}(g_t + \mu(P_0 - y_t^2)).$$

Here g_t is the state and (x_t, P_0) is the input. Repeated limiter crossings and state-dependent power measurements provide tests of nonlinear history. Stable operating regimes provide a complementary test of how much short-history approximation suffices.

Iterative optimization

For the objective $\frac{1}{2}\|Az - b\|^2 + \rho\|z\|_1$, ISTA [33] repeatedly applies

$$z_{j+1} = \operatorname{soft}_{\eta\rho}\big((I - \eta A^T A)z_j + \eta A^T b\big),$$

where $\operatorname{soft}_{\tau(a)} = \operatorname{sign}(a)\max(|a| - \tau, 0)$ acts componentwise. The iterate is the state; each optimization step combines a gradient update with soft-thresholding. FISTA [34] adds momentum:

$$z_{j+1} = \operatorname{soft}_{\eta\rho}\big(y_j - \eta A^{T(Ay_j - b)}\big), \quad y_{j+1} = z_{j+1} + \beta_{j(z_{j+1}-z_j)}.$$

Its state includes the previous iterate and the momentum schedule or clock. A surrogate study could compare accuracy and latency while the active support changes repeatedly. After support and signs stabilize, soft-thresholding is affine on the visited region, providing a useful second regime for the comparison.

Conversation and perception

An ongoing conversation can revise a referent, correct an earlier fact, or change a goal. A stream of observations can similarly revise a motion estimate. These settings suggest learning an internal state whose update incorporates each new input into the current interpretation. Textual scratchpads expose intermediate computation in generated form [1], [2]. Nonlinear carried memory offers a complementary place for that computation between observations and outputs. A useful study would vary the frequency of corrections and measure whether later responses use the revised information, alongside state size, generated-token budget, and response latency.

Driven reservoirs

A standard echo-state network [35] uses

$$h_{t+1} = \tanh(Wh_t + Uu_t + b).$$

With recurrent parameters fixed, E97 also defines a driven nonlinear dynamical system, with state mixing determined by its input keys and gates. Training such a model on long-horizon reservoir trajectories would test how well it retains consequential nonlinear history. Varying contraction and the rate of forgetting the initial condition would connect the comparison to the echo-state property.

Biological dynamics

Genomic function depends on nonlinear interactions among sequence features, regulatory signals, and cellular state. Enformer provides one example of using a nonlinear sequence model to integrate distal regulatory information for cell-type-specific predictions [36]. Pangenomic datasets run to terabases per study [37], [38], [39], motivating efficient nonlinear recurrence

for long, structured inputs. A future study could ask whether recurrent memory can integrate sequence features with changing regulatory signals and cellular context, evaluated through held-out expression or perturbation-response prediction.

Population dynamics

May's logistic population model [40] is

$$n_{t+1} = r n_{t(1-n_t)}.$$

The normalized population n_t is the state and r is a fixed parameter. An environmentally driven extension can use r_t as an input. Comparing learned trajectories across contracting, periodic, and chaotic regimes would test how error growth and history retention shape recurrent-model performance. The evaluation horizon and error metric should reflect each regime's predictability.

These tasks put the computational idea to work: learn a state update whose output remains consequential over many iterations. Width-axis multi-programming offers a practical way to run many such learned programs concurrently. Task-specific comparisons across horizon, precision, state size, and latency would establish where that design is useful.

9 Related Work

Intermediate computation in language models

Scratchpad training [1] and chain-of-thought prompting [2] use generated intermediate steps to improve multi-step computation and reasoning. The generated sequence carries the working calculation forward through successive decoding steps. Emender studies ongoing computation in a fixed-size recurrent state, updated as each input arrives. The two forms of computation can work together: generated intermediates provide an explicit working sequence, while nonlinear state updates carry a revisable internal computation through the input stream. This organization motivates tests on iterated maps and feedback tasks with a per-step time budget.

Ancestry: the fast-weight line on delta correction

The Emender update belongs to a long line that treats memory as a structure updated by prediction error. The earliest formulation is Widrow and Hoff's least-mean-square rule [41], which writes the error between target and current output back into the weight vector of an adaptive linear element. Schmidhuber recast the same idea in a recurrent setting under the name *fast-weight programmers* [42]: a slow controller emits keys and values that update a fast-weight matrix online, with the update being a correction against what the matrix currently predicts at the addressed slot. Schlag, Irie, and Schmidhuber made the bridge to modern attention explicit [43], showing that linear-transformer kernels with cumulative outer-product updates *are* fast-weight programmers with a raw additive write, and proposing the delta-rule write as a stability and capacity fix. DeltaNet [14] carried the delta rule into a parallelizable linear-recurrent language model, demonstrating the rule at billion-parameter scale in the linear-state regime. Emender combines delta correction with a nonlinear matrix-state update and width-axis multi-programming. Its saturating update also motivates the conditional latching analysis in Section 7.

Linear-state recurrent language models

The dominant comparison cohort is the family of linear-state billion-parameter recurrent LMs surveyed in Section 2: Mamba [3] and Mamba2 [4] (selective SSMs); RetNet [44] (decayed linear attention); GLA [13] (gated linear attention); DeltaNet [14] (delta rule, linear in S); Gated DeltaNet [5] (gated delta rule); RWKV-4/5/6/7 [45], [46] (linear-state WKV / generalized delta rule with linear state); HGRN2 [47] (gated linear RNN with state expansion); MinGRU/ MinLSTM [15] (input-only gates, linear scan); mLSTM/xLSTM-7B [48] (covariance update); Griffin/RecurrentGemma [16] (RG-LRU, often hybridized with local attention). Circuit-containment results for restricted SSM families require their stated transition and arithmetic assumptions [17]. The current empirical comparison uses GDN2. Its long-horizon S_5 result in Section 6 demonstrates strong learning of permutation composition, a task with an exact linear-state representation.

In the broader SSM/sequence-model lineage, S5 [49] introduces simplified state-space layers with a multi-input multi-output linear state-space recurrence. It is distinct from the permutation group S_5 used as a probe here.

A newer linear-state result, Gated DeltaNet-2 [6], decouples channel-wise erase and write gates and is reported by its authors to outperform GDN. It extends retrieval and state-tracking behavior inside the linear-state envelope, and it is the linear-recurrent baseline of this paper's 1.3 B pairing: the GDN2 arm of Section 5 is the official-style Gated DeltaNet-2 mixer (external source commit `95709fc`) with the same SwiGLU MLP as E97, selected by the same CMA-ES protocol. The E97 split gates of Section 3 are the Emender's corresponding edit — key-axis erase/read plus value-axis write — inside a *nonlinear* state map, so the two architectures are adjacent by design (Section 7). Its decoupled gates are a natural next ablation for the multi-programmed substrate, as a bounded edit to the per-step update body.

Pure-nonlinear-recurrent peers and adjacent nonlinear-state work

M²RNN [7] (Mishra, Tan, Stoica, Gonzalez, Dao 2026) introduces nonlinear matrix-state recurrence with a raw-write update $Z = \tanh(HW + kv^T)$. It shares the central systems premise of this work — that a nonlinear recurrence is serial in time and must take its parallelism from the batch and head axes — and demonstrates that the update trains at 410 M dense and 7 B MoE scale on Nemotron-CC-v2. Its headline demonstrations use hybrid stacks that interleave attention and nonlinear recurrence. The current study instead evaluates the pure recurrent E97 mixer against a matched GDN2 control. This connects the present work to other nonlinear matrix-state designs.

xLSTM-1.3B [20] is a 7:1 mixture of mLSTM (linear) and sLSTM (nonlinear) blocks; with 87.5% of its blocks linear-state, it sits closer to the linear-state class than to the pure-nonlinear-recurrent one. It is the closest scale band among prior nonlinear-recurrent results, included here as a peer with a small nonlinear-block share. The xLSTM-7B follow-up [48] uses *only* mLSTM (linear) blocks, further from the pure-nonlinear-recurrent class still.

Titans [21] uses an MLP memory with online gradient updates and is qualitatively nonlinear-state, but it interleaves attention layers and has not been evaluated as a pure recurrent model at Pile-class scale. *Zeroth-order LSTM scaling* [50] demonstrates 1 B-scale LSTM training under a non-gradient regime, outside the standard-gradient comparison drawn here.

Classical LSTM and GRU provide the foundational gated nonlinear recurrences [12]. E97 explores a matrix-memory implementation organized around many independently addressed recurrent heads.

10 Open questions

Learning and evaluation

The current pair supplies one language-model training seed and one seed for each reported computational probe. Repeated seeds, common fixed-step budgets, and comparison of terminal with best-validation checkpoints would measure the stability of the observed gaps. A feasible matched S_5 curriculum would test the contribution of the length-512 memory limit encountered by GDN2.

Held-out tokenizer-invariant bpb and downstream evaluation are the next language-model measurements. Recovering the original CMA winner logs would also supply generation-zero fitness, spread, and total search effort for the carried-in configurations (U3.1). These records would support a matched-total-budget search comparison.

Recurrent realization

O1: realizing the group tracker. Find an E97 update configuration, initial state, and decoder that carry the running S_5 product, then prove the trajectory property over all words.

G3: realizing group actions in the constrained affine channel. The permutation-matrix construction establishes S_5 tracking in the general linear-state class. Find a representation satisfying GDN2′s split-gated decay-minus-rank-one transition and production gate and decay ranges, or characterize the algebraic obstruction. The current Lean identities allow an arbitrary decay matrix; the production restrictions define this more specific realization problem. The observed learning results establish GDN2′s empirical capability under the reported protocol; G3 asks for an exact all-words construction.

G2: state-dependent parameter schedules. Extend the formal update class to gates that read carried state, and characterize its composition and cost.

G1: the cost of generating token-dependent actions. Building on the realization question, determine the resources required by the network that supplies the per-token parameters. The four-symbol group schedule provides a concrete case for this analysis.

G1–G3 are recorded in `formal/lean/GAPS-D2.md`. Persistence also needs a production-range result: the stated hold theorem assumes $\lambda \geq 1$, while production decay lies below one. A layer-level and finite-precision analysis would connect that result to practical memory retention.

Scale and implementation

A matched 3–7 B pair would test how training loss, memory use, and computational behavior change with scale. Current-scale internal ablations would separate the effects of state nonlinearity, gates, and geometry. Direct E97 occupancy profiling, actual FLOP accounting, and paired ROCm measurements would extend the systems evidence.

The application tests in Section 8 call for long-horizon error and latency measurements in their stated regimes; feedback applications additionally call for stability testing. The bibliography's flagged publication metadata should be resolved during submission preparation.

11 Conclusion

Emender makes recurrent memory a site of ongoing computation. Each input corrects the stored state and transforms what the next step receives. Running many small heads in parallel makes this sequential computation practical: E97 trains through 150.8 B Pile tokens at 1.3 B parameters and reaches comparable training loss to GDN2. A Frontier continuation extends the same architecture to 513.0 B accepted tokens, with the recorded rank-0 loss falling to 2.2798 nats/token.

The iterated nonlinear-map experiment connects that practicality to the architectural motivation. E97 learns the recurrence more accurately and retains a larger fraction of its accuracy as sequence length grows. GDN2 learns S_5 permutation composition strongly and leads its long-horizon comparison. Since this task admits an exact linear-state representation, GDN2′s result is a meaningful success on a computation compatible with linear-state recurrence. Alongside E97′s advantage on iterated nonlinear maps, it shows why the computational structure of the task matters. The formal state-channel results make the architectural distinction precise: repeated nonlinear composition in E97, and an affine GDN fold under a fixed, state-independent schedule.

These findings support studying nonlinear memory as a computational resource. The next step is to test that resource across training budgets, scales, and tasks in which an evolving state must keep being transformed: feedback, iterative optimization, and driven dynamics.

Reproducibility

The checkpoints, loading code, and measurement scripts behind the 1.3 B pairing are public: the two immutable per-arm authorities are published as E97 (`https://huggingface.co/spinozans/emender-e97-1.3b`, tag `tokens-150793748480`, the 150.79 B-token checkpoint of `emender_E97_1.3B_20260709_084606`, source .pt checkpoint SHA-256 `0239706e1f67e4823008a3a2754894b5b94dc1663580d2e40c1c74f7dd6a72b2`) and GDN2 (`https://huggingface.co/spinozans/gdn2-mlp-1.3b`, tag `tokens-152280498176`, source .pt checkpoint SHA-256 `ffae61cfeeeff820f469f7d66dd31c60e48c455872ff9102e57b15ad75bad59b`). The hashes above identify the source `.pt` checkpoints in the release provenance. The immutable HF revisions are `119c6623f414aa7fc6e101667d3ad89b7ed6af73` (E97) and `e6087fa6984265404d99c419c095692d840da472` (GDN2). Converted-weight digests are in each revision's `SHA256SUMS`. The releases ship with the per-architecture CMA-ES configurations, the training protocol (launcher manifests in `scripts/launch_emender_8gpu_diloco.sh` and `scripts/launch_gdn2_mlp_8gpu_diloco.sh`), and the Triton multi-programming kernel source in the public `emender` repo; the canonical paper PDF is mirrored at `http://hypervolu.me/~erik/ndm/Garrison_2026_Emender.pdf`. The Frontier continuation's retained-checkpoint lineage, accepted-token anchors, and the job-5134243 communication measurements are recorded in `docs/validation/`, `reports/frontier/`, and the corrected 513 B audit.

12 Next tests for the current pair

The current results motivate three direct tests. First, repeat the iterated-map and S_5 protocols across seeds and common realized training budgets, recording both terminal and best-validation checkpoints. Persistence of the map advantage would strengthen the mechanism hypothesis; its disappearance under those controls would locate an optimization or protocol dependence. Second, ablate the state nonlinearity within the current E97 geometry, holding the remaining implementation and training choices fixed, to test its causal role at this scale. Third, evaluate the two immutable language-model checkpoints on held-out text and downstream tasks.

13 Future Work

A matched 3–7 B E97/GDN2 study would test training viability, memory cost, and computational behavior across scale. The feedback, optimization, and driven-dynamics tasks in Section 8 would test utility beyond language. Mathematical targets are a recurrent tanh tracker (O1), constrained affine/reflection realizability (G3), production-range persistence, and finite-precision robustness.

References

[1] M. Nye *et al.*, "Show Your Work: Scratchpads for Intermediate Computation with Language Models," *arXiv preprint arXiv:2112.00114*, 2021, doi: 10.48550/arXiv.2112.00114.

[2] J. Wei *et al.*, "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models," *arXiv preprint arXiv:2201.11903*, 2022, doi: 10.48550/arXiv.2201.11903.

[3] A. Gu and T. Dao, "Mamba: Linear-Time Sequence Modeling with Selective State Spaces," *arXiv preprint arXiv:2312.00752*, 2024.

[4] T. Dao and A. Gu, "Transformers are SSMs: Generalized Models and Efficient Algorithms Through Structured State Space Duality," *arXiv preprint arXiv:2405.21060*, 2024.

[5] S. Yang, J. Kautz, and A. Hatamizadeh, "Gated Delta Networks: Improving Mamba2 with Delta Rule," *arXiv preprint arXiv:2412.06464*, 2024.

[6] A. Hatamizadeh, Y. Choi, and J. Kautz, "Gated DeltaNet-2: Decoupling Erase and Write in Linear Attention," *arXiv preprint arXiv:2605.22791*, 2026.

[7] M. Mishra, S. Tan, I. Stoica, J. Gonzalez, and T. Dao, "M²RNN: Non-Linear RNNs with Matrix-Valued States for Scalable Language Modeling." 2026.

[8] F. Danieli, P. Rodriguez, M. Sarabia, X. Suau, and L. Zappella, "ParaRNN: Unlocking Parallel Training of Nonlinear RNNs for Large Language Models," *arXiv preprint arXiv:2510.21450*, 2025.

[9] L. Gao *et al.*, "The Pile: An 800GB Dataset of Diverse Text for Language Modeling," *arXiv preprint arXiv:2101.00027*, 2020.

[10] A. Vaswani *et al.*, "Attention Is All You Need," *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[11] J. L. Elman, "Finding Structure in Time," *Cognitive Science*, vol. 14, no. 2, pp. 179–211, 1990, doi: 10.1207/s15516709cog1402_1.

[12] S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997, doi: 10.1162/neco.1997.9.8.1735.

[13] S. Yang, B. Wang, Y. Shen, R. Panda, and Y. Kim, "Gated Linear Attention Transformers with Hardware-Efficient Training," *arXiv preprint arXiv:2312.06635*, 2023.

[14] S. Yang, B. Wang, Y. Zhang, Y. Shen, and Y. Kim, "Parallelizing Linear Transformers with the Delta Rule over Sequence Length," *arXiv preprint arXiv:2406.06484*, 2024.

[15] L. Feng, F. Tung, M. O. Ahmed, Y. Bengio, and H. Hajimirsadeghi, "Were RNNs All We Needed?," *arXiv preprint arXiv:2410.01201*, 2024.

[16] S. De *et al.*, "Griffin: Mixing Gated Linear Recurrences with Local Attention for Efficient Language Models," *arXiv preprint arXiv:2402.19427*, 2024.

[17] W. Merrill, J. Petty, and A. Sabharwal, "The Illusion of State in State-Space Models," *arXiv preprint arXiv:2404.08819*, 2024.

[18] W. Merrill and A. Sabharwal, "A Logic for Expressing Log-Precision Transformers," in *Advances in Neural Information Processing Systems*, 2023. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2023/hash/a48e5877c7bf86a513950ab23b360498-Abstract-Conference.html

[19] D. Chiang, "Transformers in Uniform TC0," *Transactions on Machine Learning Research*, 2025, [Online]. Available: https://openreview.net/forum?id=ZA7D4nQuQF

[20] M. Beck *et al.*, "xLSTM: Extended Long Short-Term Memory," *arXiv preprint arXiv:2405.04517*, 2024.

[21] A. Behrouz, P. Zhong, and V. Mirrokni, "Titans: Learning to Memorize at Test Time," *arXiv preprint arXiv:2501.00663*, 2025.

[22] H. T. Siegelmann and E. D. Sontag, "On the Computational Power of Neural Nets," *Journal of Computer and System Sciences*, vol. 50, no. 1, pp. 132–150, 1995, doi: 10.1006/jcss.1995.1013.

[23] D. A. M. Barrington, "Bounded-Width Polynomial-Size Branching Programs Recognize Exactly Those Languages in NC^1", in *Proceedings of the Eighteenth Annual ACM Symposium on Theory of Computing (STOC 1986)*, ACM, 1986, pp. 1–5. doi: 10.1145/12130.12131.

[24] P. Tillet, H. T. Kung, and D. Cox, "Triton: An Intermediate Language and Compiler for Tiled Neural Network Computations," *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*, pp. 10–19, 2019, doi: 10.1145/3315508.3329973.

[25] A. Defazio, X. A. Yang, H. Mehta, K. Mishchenko, A. Khaled, and A. Cutkosky, "The Road Less Scheduled," *arXiv preprint arXiv:2405.15682*, 2024.

[26] A. Douillard *et al.*, "DiLoCo: Distributed Low-Communication Training of Language Models," *arXiv preprint arXiv:2311.08105*, 2023.

[27] L. de Moura and S. Ullrich, "The Lean 4 Theorem Prover and Programming Language," in *Automated Deduction – CADE 28*, 2021. doi: 10.1007/978-3-030-79876-5_37.

[28] The Mathlib Community, "Mathlib 4: Library Documentation." [Online]. Available: https://leanprover-community.github.io/mathlib4_docs/

[29] H. K. Khalil, *Nonlinear Systems*, 3rd ed. Prentice Hall, 2002.

[30] J. C. Candy and G. C. Temes, Eds., *Oversampling Delta-Sigma Data Converters: Theory, Design, and Simulation*. IEEE Press, 1992.

[31] S. Haykin, *Adaptive Filter Theory*, 4th ed. Prentice Hall, 2002.

[32] R. G. Lyons, *Understanding Digital Signal Processing*, 3rd ed. Prentice Hall, 2011.

[33] I. Daubechies, M. Defrise, and C. De Mol, "An Iterative Thresholding Algorithm for Linear Inverse Problems with a Sparsity Constraint," *Communications on Pure and Applied Mathematics*, vol. 57, no. 11, pp. 1413–1457, 2004, doi: 10.1002/cpa.20042.

[34] A. Beck and M. Teboulle, "A Fast Iterative Shrinkage-Thresholding Algorithm for Linear Inverse Problems," *SIAM Journal on Imaging Sciences*, vol. 2, no. 1, pp. 183–202, 2009, doi: 10.1137/080716542.

[35] H. Jaeger, "The "Echo State" Approach to Analysing and Training Recurrent Neural Networks," technical report 148, 2001. [Online]. Available: https://www.ai.rug.nl/minds/uploads/EchoStatesTechRep.pdf

[36] Ž. Avsec *et al.*, "Effective gene expression prediction from sequence by integrating long-range interactions," *Nature Methods*, vol. 18, pp. 1196–1203, 2021, doi: 10.1038/s41592-021-01252-x.

[37] W.-W. Liao *et al.*, "A draft human pangenome reference," *Nature*, vol. 617, no. 7960, pp. 312–324, 2023, doi: 10.1038/s41586-023-05896-x.

[38] A. Guarracino *et al.*, "Recombination between heterologous human acrocentric chromosomes," *Nature*, vol. 617, no. 7960, pp. 335–343, 2023, doi: 10.1038/s41586-023-05976-y.

[39] E. Garrison *et al.*, "Building pangenome graphs," *Nature Methods*, vol. 21, pp. 2008–2012, 2024, doi: 10.1038/s41592-024-02430-3.

[40] R. M. May, "Simple Mathematical Models with Very Complicated Dynamics," *Nature*, vol. 261, pp. 459–467, 1976, doi: 10.1038/261459a0.

[41] B. Widrow and M. E. Hoff, "Adaptive Switching Circuits," in *IRE WESCON Convention Record, Part 4*, 1960, pp. 96–104. [Online]. Available: https://isl.stanford.edu/~widrow/papers/c1960adaptiveswitching.pdf

[42] J. Schmidhuber, "Learning to Control Fast-Weight Memories: An Alternative to Dynamic Recurrent Networks," *Neural Computation*, vol. 4, no. 1, pp. 131–139, 1992, doi: 10.1162/neco.1992.4.1.131.

[43] I. Schlag, K. Irie, and J. Schmidhuber, "Linear Transformers Are Secretly Fast Weight Programmers," in *Proceedings of the 38th International Conference on Machine Learning (ICML)*, in Proceedings of Machine Learning Research, vol. 139. 2021, pp. 9355–9366.

[44] Y. Sun *et al.*, "Retentive Network: A Successor to Transformer for Large Language Models," *arXiv preprint arXiv:2307.08621*, 2023.

[45] B. Peng and others, "RWKV: Reinventing RNNs for the Transformer Era," *arXiv preprint arXiv:2305.13048*, 2023.

[46] B. Peng and others, "RWKV-7 "Goose" with Expressive Dynamic State Evolution," *arXiv preprint arXiv:2503.14456*, 2025.

[47] Z. Qin *et al.*, "HGRN2: Gated Linear RNNs with State Expansion," *arXiv preprint arXiv:2404.07904*, 2024.

[48] M. Beck and others, "xLSTM-7B: A Recurrent LLM for Fast and Efficient Inference," *arXiv preprint arXiv:2503.13427*, 2025.

[49] J. T. H. Smith, A. Warrington, and S. W. Linderman, "Simplified State Space Layers for Sequence Modeling," *arXiv preprint arXiv:2208.04933*, 2022.

[50] F. Chaubard and M. J. Kochenderfer, "Scaling Recurrent Neural Networks to a Billion Parameters with Zero-Order Optimization," *arXiv preprint arXiv:2505.17852*, 2025.

APPENDIX A Development history and source register

The project developed through successive recurrent-memory designs, implementation changes, and increasingly controlled comparisons. This register records that process and points to its source material. The empirical study in this paper is the current E97/GDN2 pair; the entries below preserve the development trail.

The source record remains available for provenance:

- *Architecture development.* `paper/main.typ` at base commit `f17b3be20b395cef914cd7034e8691c997164230` records the E63-to-E97 development sequence, ablations, and implementation decisions. It records how the program evolved.
- *Exploratory task and search work.* `experiments/expressivity_tasks/`, `experiments/grok_symmetric_width/`, `paper/ndmpapernotes.md`, and `paper/results/cmaes_burst_v2/` retain development records. These records identify their own protocols and model versions.
- *The present comparison.* The two immutable model releases and launcher manifests in the reproducibility section identify the actual E97 and GDN2 arms. `paper/results/expressivity_1p3b/` contains their computational-probe records. The current-pair tables in this paper read the corresponding result JSONs directly.
- *Formal development.* `formal/lean/` retains the full proof project. The theorem names cited in Section 7 identify the specific statements used here.

The original records remain available in the repository. The present comparison is defined by its own model identities, data, and protocols.